Sierra 최적화 과정

개요

Sierra 중간표현에서 수행되는 최적화는 Cairo 프로그램의 성능과 가스 효율성을 크게 좌우하는 핵심 단계입니다. 이 문서에서는 Sierra 최적화의 주요 구성요소인 가스 계산 최적화, AP 변화 추적, 타입 크기 분석, 데드 코드 제거 등의 메커니즘을 상세히 분석합니다.

가스 계산 최적화 기법

가스 계산 시스템 구조

Cairo 컴파일러는 두 단계의 가스 계산을 수행합니다:

1. **Pre-cost 계산**: 비-step 토큰들의 비용 계산
2. **Post-cost 계산**: Step 토큰의 비용 계산

```
// 가스 정보 구조체
pub struct GasInfo {
/// 각 statement와 토큰 타입별 변수 값
pub variable_values: OrderedHashMap<(StatementIdx, CostTokenType), i64>,
/// 함수별 가스 비용
pub function_costs: OrderedHashMap<FunctionId, CostTokenMap<i64>>,
}
```

비용 토큰 타입

Sierra는 다양한 비용 토큰을 사용하여 세밀한 가스 계산을 수행합니다:

```
pub enum CostTokenType {
Const, // 상수 비용 (steps, holes, range_checks 등)
Pedersen, // Pedersen 해시 builtin
Poseidon, // Poseidon 해시 builtin
EcOp, // 타원곡선 연산 builtin
Bitwise, // 비트 연산 builtin
AddMod, // 모듈러 덧셈 (회로용)
MulMod, // 모듈러 곱셈 (회로용)
}
```

가스 최적화 전략

1. 방정식 기반 최적화

컴파일러는 각 statement의 가스 비용을 방정식으로 모델링하고 선형 프로그래밍을 통해 최적해를 찾습니다:

```
// 가스 방정식 생성 및 해결
let equations = generate_equations::generate_equations(program, get_cost)?;
let solution = cairo_lang_eq_solver::try_solve_equations(
token_equations,
minimization_vars
).ok_or(CostError::SolvingGasEquationFailed)?;
```

2. 최소화 변수 우선순위

가스 최적화는 다음 우선순위로 변수를 최소화합니다:

1. **함수 비용 변수**: 다른 곳에서도 호출될 수 있는 함수의 비용 우선 최소화
2. **가스 인출 변수**: 사용자에게 직접 비용이 부과되는 `withdraw_gas` 변수
3. **브랜치 정렬 변수**: `branch_align` 에서 소모되는 가스 변수

```
let mut minimization_vars = vec![vec![], vec![], vec![]];
for v in token_equations.iter().flat_map(|eq| eq.var_to_coef.keys()).unique()
minimization_vars[match v {
Var::LibfuncImplicitGasVariable(idx, _) => {
match registry.get_libfunc(&invocation.libfunc_id).unwrap() {
CoreConcreteLibfunc::BranchAlign(_) => 2,
CoreConcreteLibfunc::Gas(GasConcreteLibfunc::WithdrawGas(_)) => 1,
CoreConcreteLibfunc::Gas(GasConcreteLibfunc::BuiltinWithdrawGas(_)) => 0,
_ => unreachable!(),
}
}
Var::StatementFuture(idx, _) if non_set_cost_func_entry_points.contains(idx) =>
_ => continue,
}].push(v.clone())
}
```

3. 지갑(Wallet) 값 최적화

각 statement에서 필요한 최소 가스량을 "지갑 값"으로 모델링하여 최적화합니다:

```
pub struct WalletInfo<CostType: CostTypeTrait> {
/// statement 실행 전 필요한 최소 지갑 값
value: CostType,
}

impl<CostType: CostTypeTrait> WalletInfo<CostType> {
fn merge(
branch_costs: &[BranchCost],
branches: Vec<Self>,
target_value: Option<&CostType>,
) -> Self {
let mut max_value = CostType::max(
branches.iter().map(|wallet_info| wallet_info.value.clone())
);
// 브랜치가 여러 개이거나 redeposit_gas인 경우 target_value까지 증가 가능
if (is_branch_align || is_redeposit) && let Some(target_value) = target_value {
max_value = CostType::max([max_value, target_value.clone()].into_iter());
}
WalletInfo { value: max_value }
}
}
```

AP 변화 추적 메커니즘

AP 추적 시스템

AP(Allocation Pointer) 변화 추적은 Cairo VM의 메모리 관리 최적화를 위한 핵심 메커니즘입니다.

```
#[derive(Clone, Debug, Eq, PartialEq)]
pub  enum ApChange {
Unknown, // 알 수 없는 변화
Known(usize), // 알려진 크기의 변화
FromMetadata, // 메타데이터에서 제공되는 변화
AtLocalsFinalization(usize), // 로컬 변수 완료 시점의 변화
FunctionCall(FunctionId), // 함수 호출에 따른 변화
FinalizeLocals, // 로컬 변수 할당 완료
EnableApTracking, // AP 추적 활성화
DisableApTracking, // AP 추적 비활성화
}
```

AP 추적 정보 구조

```
struct ApTrackingInfo {
/// 추적 기준점
base: ApTrackingBase,
/// 기준점으로부터의 AP 변화량
ap_change: usize,
```

```
}

enum ApTrackingBase {
FunctionStart(FunctionId), // 함수 시작점
EnableStatement(StatementIdx), // AP 추적 활성화 지점
}
```

AP 변화 계산 알고리즘

1. 로컬 변수 크기 계산

```
fn calc_locals_for_statement(&mut self, idx: StatementIdx) -> Result<(), ApCha
for (ap_change, target) in self.get_branches(idx)? {
match ap_change {
ApChange::AtLocalsFinalization(x) => {
self.locals_size.insert(target, self.get_statement_locals(idx) + x);
}
ApChange::FinalizeLocals => {}
_ => {
if let Some(locals) = self.locals_size.get(&idx) {
self.locals_size.insert(target, *locals);
}
}
}
```

```
}
Ok(())
}
```

2. 함수 AP 변화 계산

```
fn calc_known_ap_change_for_statement(&mut self, idx: StatementIdx) -> Result<
let mut max_change = 0;
for (ap_change, target) in self.get_branches(idx)? {
let Some(target_ap_change) = self.known_ap_change_to_return.get(&target) else {
return Ok(());
};
if let Some(ap_change) = self.branch_ap_change(idx, &ap_change, |id| {
self.known_ap_change_to_return.get(&self.func_entry_point(id).ok()?).cloned()
}) {
max_change = max_change.max(target_ap_change + ap_change);
} else {
return Ok(());
};
}
self.known_ap_change_to_return.insert(idx, max_change);
Ok(())
```

```
}
```

3. 브랜치별 AP 변화 계산

```
fn branch_ap_change(
&self,
idx: StatementIdx,
ap_change: &ApChange,
func_ap_change: impl Fn(&FunctionId) -> Option<usize>,
) -> Option<usize> {
match ap_change {
ApChange::Unknown | ApChange::DisableApTracking => None,
ApChange::Known(x) => Some(*x),
ApChange::FromMetadata | ApChange::AtLocalsFinalization(_) | ApChange::EnableApT
ApChange::FinalizeLocals => Some(self.get_statement_locals(idx)),
ApChange::FunctionCall(id) => func_ap_change(id).map(|x| 2 + x), // 함수 호출 오버
}
}
```

타입 크기 분석 과정

타입 크기 맵 생성

Sierra는 모든 타입의 메모리 크기를 사전에 계산하여 최적화에 활용합니다:

```
pub type TypeSizeMap = UnorderedHashMap<ConcreteTypeId, i16>;


pub fn get_type_size_map(
program: &Program,
registry: &ProgramRegistry<CoreType, CoreLibfunc>,
) -> Result<TypeSizeMap, Box<ProgramRegistryError>> {
let mut type_sizes = TypeSizeMap::default();
for declaration in &program.type_declarations {
let size = match registry.get_type(&declaration.id)? {
// 크기 0
CoreTypeConcrete::Coupon(_) => 0,
// 크기 1
CoreTypeConcrete::Felt252(_) | CoreTypeConcrete::Uint128(_) |
CoreTypeConcrete::Box(_) | CoreTypeConcrete::RangeCheck(_) => 1,
// 크기 2
CoreTypeConcrete::Array(_) | CoreTypeConcrete::Span(_) |
CoreTypeConcrete::EcPoint(_) => 2,
// 가변 크기
CoreTypeConcrete::Struct(struct_type) => {
let mut size: i16 = 0;
for member in &struct_type.members {
size = size.checked_add(get_dep_size(member)?).ok_or_else(size_overflow_error)?;
```

```
}
size
}
CoreTypeConcrete::Enum(enum_type) => {
let mut max_variant_size: i16 = 0;
for variant in &enum_type.variants {
max_variant_size = max_variant_size.max(get_dep_size(variant)?);
}
max_variant_size.checked_add(1).ok_or_else(size_overflow_error)? // 태그 크기 추가
}
_ => continue, // 저장되지 않는 타입들
};
type_sizes.insert(declaration.id.clone(), size);
}
Ok(type_sizes)
}
```

타입 크기 기반 최적화

1. 배열 연산 최적화

```
ArrayConcreteLibfunc::Append(libfunc) => {
vec![ConstCost::steps(info_provider.type_size(&libfunc.ty) as i32).into()]
}
```

```
ArrayConcreteLibfunc::Get(libfunc) => {
if  info_provider.type_size(&libfunc.ty) == 1 {
vec![
(ConstCost::steps(5) + ConstCost::range_checks(1)).into(),
(ConstCost::steps(5) + ConstCost::range_checks(1)).into(),
]
} else {
vec![
(ConstCost::steps(6) + ConstCost::range_checks(1)).into(),
(ConstCost::steps(6) + ConstCost::range_checks(1)).into(),
]
}
}
```

2. 메모리 연산 최적화

```
Mem(libfunc) => match  libfunc {
StoreTemp(libfunc) => {
vec![ConstCost::steps(info_provider.type_size(&libfunc.ty) as i32).into()]
}
StoreLocal(libfunc) => {
let  size = info_provider.type_size(&libfunc.ty) as i32;
vec![(ConstCost::steps(size) + ConstCost::holes(-size)).into()]
```

```
}
AllocLocal(libfunc) => {
vec![ConstCost::holes(info_provider.type_size(&libfunc.ty) as i32).into()]
}
_ => vec![ConstCost::default().into()],
}
```

데드 코드 제거 최적화 분석

도달 가능성 분석

Sierra 최적화는 위상 정렬을 통해 도달 불가능한 코드를 식별합니다:

```
fn compute_reverse_topological_order(
n_statements: usize,
detect_cycles: bool,
dependencies_callback: impl Fn(&StatementIdx) -> Vec<StatementIdx>,
) -> Result<Vec<StatementIdx>, CostError> {
reverse_topological_ordering(
detect_cycles,
(0..n_statements).map(StatementIdx),
n_statements,
|idx| Ok(dependencies_callback(&idx)),
|_| CostError::UnexpectedCycle,
```

```
)

}
```

사용되지 않는 함수 제거

```
let  non_set_cost_func_entry_points: UnorderedHashSet<_> = program

.funcs

.iter()

.filter(|f| !function_set_costs.contains_key(&f.id))

.map(|f| f.entry_point)

.collect();
```

호출되지 않는 함수들은 가스 계산에서 제외되어 효과적으로 제거됩니다.

불필요한 가스 연산 제거

```
// withdraw_gas와 refund libfunc의 변수를 0으로 설정

for (i, statement) in  program.statements.iter().enumerate() {

let Statement::Invocation(invocation) = statement  else { continue; };

let Ok(libfunc) = program_info.registry.get_libfunc(&invocation.libfunc_id) else

let  is_withdraw_gas = matches!(libfunc, CoreConcreteLibfunc::Gas(GasConcreteLib

let  is_refund = matches!(libfunc, CoreConcreteLibfunc::Coupon(CouponConcreteLib

if  is_withdraw_gas || is_refund {

for  token  in CostTokenType::iter_precost() {
```

```
assert_eq!(info.variable_values.insert((StatementIdx(i), *token), 0), None);
}
}
}
```

성능 최적화 전략

1. 딕셔너리 최적화

```
// 딕셔너리 스쿼시 비용 상수들
pub const DICT_SQUASH_UNIQUE_KEY_COST: ConstCost =
ConstCost { steps: 46, holes: 0, range_checks: 6, range_checks96: 0 };
pub const DICT_SQUASH_REPEATED_ACCESS_COST: ConstCost =
ConstCost { steps: 9, holes: 0, range_checks: 1, range_checks96: 0 };
pub const DICT_SQUASH_FIXED_COST: ConstCost =
ConstCost { steps: 57, holes: 0, range_checks: 3, range_checks96: 0 };
```

딕셔너리 접근 시 고유 키에 대해서는 높은 비용을, 반복 접근에 대해서는 낮은 비용을 부과하여 효율적인 사용을 유도합니다.

2. 세그먼트 아레나 최적화

```
pub const SEGMENT_ARENA_ALLOCATION_COST: ConstCost =
```

```
ConstCost { steps: 8, holes: 0, range_checks: 0, range_checks96: 0 };
```

세그먼트 할당 시 미리 완료 비용을 부과하여 메모리 관리를 최적화합니다.

3. 회로 연산 최적화

```
CircuitConcreteLibfunc::Eval(libfunc) => {
let  info = info_provider.circuit_info(&libfunc.ty);
let  instance_size: i32 = MOD_BUILTIN_INSTANCE_SIZE.into_or_panic();
let  mut  steps: i32 = 8 + instance_size;
if !info.add_offsets.is_empty() {
steps += instance_size;
}
vec![
BranchCost::Regular {
const_cost: ConstCost::steps(steps),
pre_cost: PreCost(CostTokenMap::from_iter([
(CostTokenType::AddMod, info.add_offsets.len().into_or_panic()),
(CostTokenType::MulMod, info.mul_offsets.len().into_or_panic()),
])),
},
// Success case도 동일한 비용
]
}
```

최적화 결과 검증

1. 강제 지갑 값 검증

```
for (idx, value) in  enforced_wallet_values.iter() {
if  context.wallet_at_ex(idx, false).value != *value {
return Err(CostError::EnforceWalletValueFailed(*idx));
}
}
```

2. AP 변화 일관성 검증

```
match  self.tracking_info.entry(target) {
Entry::Occupied(e) => {
e.into_mut().ap_change = e.get().ap_change.max(base_info.ap_change);
}
Entry::Vacant(e) => {
e.insert(base_info);
}
}
```

결론

Sierra 최적화는 가스 효율성, 메모리 사용량, 실행 성능을 종합적으로 고려한 정교한 시스템입니다. 방정식 기반 가스 계산, AP 변화 추적, 타입 크기 분석, 데드 코드 제거 등의 기법을 통해 Cairo 프로그램의 성능을 크게 향상시킵니다. 이러한 최적화는 특히 블록체인 환경에서 가스 비용이 직접적인 경제적 영향을 미치는 상황에서 매우 중요한 역할을 합니다.